集群操作手册 For YHC V1.0

created by Han

Last Updated:2023/08/23

集群操作手册 For YHC V1.0
- 基础教学
  - 首先确定自己所处的网路环境
  - 登录集群（SSH登录）
  - 操作
- 集群计算
  - 常规计算
    - mpirun(🍊多半看不懂，可以跳过的)
    - PBS（这个要掌握@YHC
  - 计算实例——Han常用的Jupyter 云计算
    - 完整全流程，适用于长时间计算
    - 便捷快速连接Jupyter（利用集群调试节点），适用于短时间调试和计算

基础教学

首先确定自己所处的网路环境

集群是受特殊保护的计算资源，是属于学校的内网资源，严格上的来说只允许校内教育网络访问。我们在没有校内网络环境的时候可以通过VPN访问。登陆地址：10.130.2.222 端口：22

综上所述：

- 如果在校内需要登陆，需要登陆'OUC-AUTO'或'OUC-WIFI'，或者使用校园有线网络。
- 如果在校外（或无法登录校园网）登录，需要在信息门户申请集群的IP地址，再登陆atrust来连接集群。

登录集群（SSH登录）

SSH信息	值
集群登陆节点IP	10.130.2.222
SSH 登录端口	22
登陆软件	Termianl，Putty，MobaXterm等等终端（终端模拟器）均可

- 一般情况下需要输入密码，如果本机配置了免密凭证可以免密登录。
- 密码输入时会不可见，但是已经输入上了。

操作

常规Linux操作，需要记得以下几点

- 我们登陆的是集群的登录节点，**不可以在你登陆的地方直接进行计算**。如需计算请参考'集群计算'一章。
- 管理员说不允许在登陆节点上运行Python（悲
- 可以使用scp命令（scp命令学习）或其余命令进行文件传输。
- 传输多数量文件时可以使用xftp。
- 保证账户安全，减少信息泄露。
- 登陆节点有时候能联网，有时候不行（迷

集群计算

本章以Python计算为基础，在开始计算前一定要明白我们在集群上的计算都是

常规计算

mpirun(🍊多半看不懂，可以跳过的)

在程序中，不同的进程需要相互的数据交换，特别是在科学计算中，需要大规模的计算与数据交换，集群可以很好解决单节点计算力不足的问题，但在集群中大规模的数据交换是很耗费时间的，因此需要一种在多节点的情况下能快速进行数据交流的标准，这就是MPI。

1. intel mpi （Fortran&C）

```
mpirun -genv I_MPI_FABRICS shm:tmi -machinefile /tmp/nodefile.$$ -n $NP
$EXEC
```

参数	含义
-genv I_MPI_DEVICE rdma	指定跑 ib 网络，rdma 可换成 rdssm
-machinefile /tmp/nodefile.$$	指定跑哪几个节点，节点由调度器分配
-n $NP	指定总共跑几个核，也是由调度器分配，默 认是均分
$EXEC	用户的应用可执行文件，后面跟输入文件等

Intel MPI编译命令及其对应关系

编译命令	调用的默认编译器命令	支持的语言
mpicc	gcc, cc	C
mpicxx	g++	C/C++
mpifc	gfortran	Fortran77* /Fortran 95*
mpigcc	gcc	C
mpigxx	g++	C/C++
mpif77	g77	Fortran 77
mpif90	gfortran	Fortran 95
mpiicc	icc	C
mpiicpc	icpc	C++
mpiifort	ifort	Fortran77/Fortran 95

更多Intel MPI请参考[开始使用适用于 Linux* 操作系统上的英特尔 oneAPI 的英特尔®® MPI 库 (intel.com)]

2. open mpi

```
mpirun --mca btl openib,self -np $nprocs -hostfile $PBS_NODEFILE $EXEC
<in.shock
```

参数	含义
--mca btl openib,self	指定跑 ib 网络
-hostfile $PBS_NODEFILE	指定跑哪几个节点，节点由调度器分配
-np $nprocs	指定总共跑几个核，也是由调度器分配，默 认是均分
$EXEC	你的应用可执行文件，后面跟输入文件等

更多请参考 [Quick start — Open MPI 5.0.x documentation (open-mpi.org)]

PBS（这个要掌握@YHC

1. 命令行操作

- qsub 用于提交脚本作业（见第二部分）（🌟）
- qstat 用于查询已经提交的作业（🌟）

```
qstat [-f][-a][-i][-n][-s][-R][-Q][-q][-B][-u]
```

参数	说明
-f	jobid 列出指定作业的详细信息（太详细了，你基本读不懂）
-a	列出自己所有作业
-i	列出不在运行的作业
-n	列出分配给此作业的结点

参数	说明
-s	列出队列管理员与 scheduler 所提供的建议
-R	列出磁盘预留信息
-Q	操作符是 destination id，指明请求的是队列状态（就是把还在排队的任务列出来）
-q	列出队列状态，并以 alternative 形式显示
-au userid	列出指定用户的所有作业
-B	列出 PBS Server 信息
-r	列出所有正在运行的作业
-Qf queue	列出指定队列的信息
-u	若操作符为作业号，则列出其状态。若操作符为 destination id， 则列出运行在其上的属于 user_list 中用户的作业状态。

- qdel：取消作业（🌟）

```
qdel [-W 间隔时间] 作业id
```

ADD：作业的状态（queue）代码有

代码	状态
C	作业结束
Q	作业排队中
R	作业运行中

- qhold：挂起作业(仅了解，不常用)

  使用 qhold 命令可以挂起作业，使其不被调度执行；

- qrls：释放挂起的作业(仅了解，不常用)

  使用 qrls 命令可以将挂起的作业释放，使其可以被调度执行；

- qrerun：重新运行作业(仅了解，不常用)
- qmove：将作业移动到另一个队列(仅了解，不常用)
- qalter： 更改作业资源属性(仅了解，不常用)

2. 编写PBS脚本并用qsub提交（🌟）

举例说明，以下是文件 `geta.pbs` 内容

```
#pbs.sh
#!/bin/bash
#PBS -N geta
#PBS -o /lustre/home/yuhanxue/InfoOut/myo3.out
```

```
#PBS -e /lustre/home/yuhanxue/InfoOut/mye3.out
#PBS -l nodes=1:ppn=4
#PBS -q com
#PBS -V
#PBS -S /bin/bash
#PBS -l walltime=1000:00:00
echo "------------------------------------"
cd /lustre/home/yuhanxue/code
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
python -V
echo "The Jobbed is $id"
NP=`cat $PBS_NODEFILE|wc -l`
echo "The np is $NP"
python geta.py
```

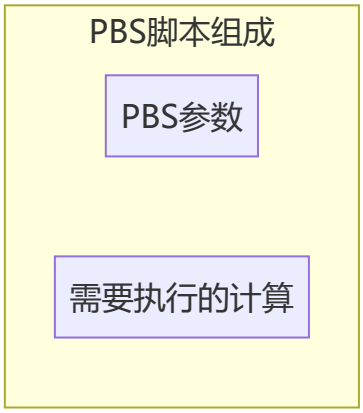
- `#PBS -N geta`：该行为作业指定了一个名字"geta"。
- `#PBS -o /lustre/home/yuhanxue/InfoOut/myo3.out`：指定标准输出文件的位置和名称。作业运行的输出将被写入此文件。
- `#PBS -e /lustre/home/yuhanxue/InfoOut/mye3.out`：指定错误输出文件的位置和名称。作业运行的错误信息将被写入此文件。
- `#PBS -l nodes=1:ppn=4`：要求PBS系统分配一个节点，并在该节点上提供4个处理器。
- `#PBS -q com`：指定作业在"com"队列中运行。
- `#PBS -V`：这个指令会导入用户的环境变量到作业。
- `#PBS -S /bin/bash`：设置作业的shell为/bin/bash。
- `#PBS -l walltime=1000:00:00`：设置作业的最大运行时间为1000小时。
- `cd /lustre/home/yuhanxue/code`：切换到存放你的代码的目录。
- `id=`echo $PBS_JOBID|awk -F. '{print $1}'`：提取PBS作业ID的前部分，保存到变量id中。
- `python -V`：显示运行作业的python的版本。
- `echo "The Jobbed is $id"`：打印作业的ID到输出文件。
- `NP=` cat $PBS_NODEFILE|wc -l`：计算并存储PBS节点文件中的行数，这通常对应了作业中使用的处理器(core)的数量。
- `echo "The np is $NP"`：打印作业中使用的处理器的数量。
- `python geta.py`：运行一份名为geta.py的Python脚本。

  执行这个文件：qsub geta.pbs

以下是完整的PBS脚本参数

参数	含义
-N 名称	定义作业的名称
-l walltime=时长	定义作业的最长运行时间
-l nodes=节点数:ppn=每节点进程数	定义作业所需的资源
-j oe	定向标准错误输出流和标准输出流到同一文件
-o 文件路径	将标准输出流重定向到指定的文件
-e 文件路径	将标准错误输出流重定向到指定的文件
-V	将环境变量传递到作业的执行环境
-q 队列名称	指定提交作业到哪个队列
-r y/n	如果作业被挂起，是否可以重新运行
-f	强制提交，即使作业所请求的资源比所在队列的最大限制要大
-p 优先级	指定作业的优先级
-a 预定时间	表示在预定时间后才能运行的作业
-c 检查点选项	表示作业在运行中可以进行检查点操作
-C 检查点目录	检查点文件的保存目录
-d 作业运行目录	作业启动的初始目录
-hold_jid 作业列表	依赖指定作业列表中的所有作业都已经成功完成后才能运行
-S 解释程序路径	执行作业的解释程序路径，如 /bin/sh, /bin/csh, /bin/bash
-u 用户名	指定执行该作业的用户名
-W 参数	可用于指定一些附加参数
-h	提交作业后，作业处于 hold 状态，即不立即执行，需要用 qrls 命令解除 hold 状态后才能执行
-k o,e,oe,n	标准输出、标准错误输出是否保存和路径
-t 数组索引	提交任务数组，数组索引可以是单一值，如1，多个值，如1,2,3，或范围，如1-10
-v 变量列表	通过 qsub 提交作业时，将指定的环境变量值传给作业
-z	不要创建未使用的输出文件

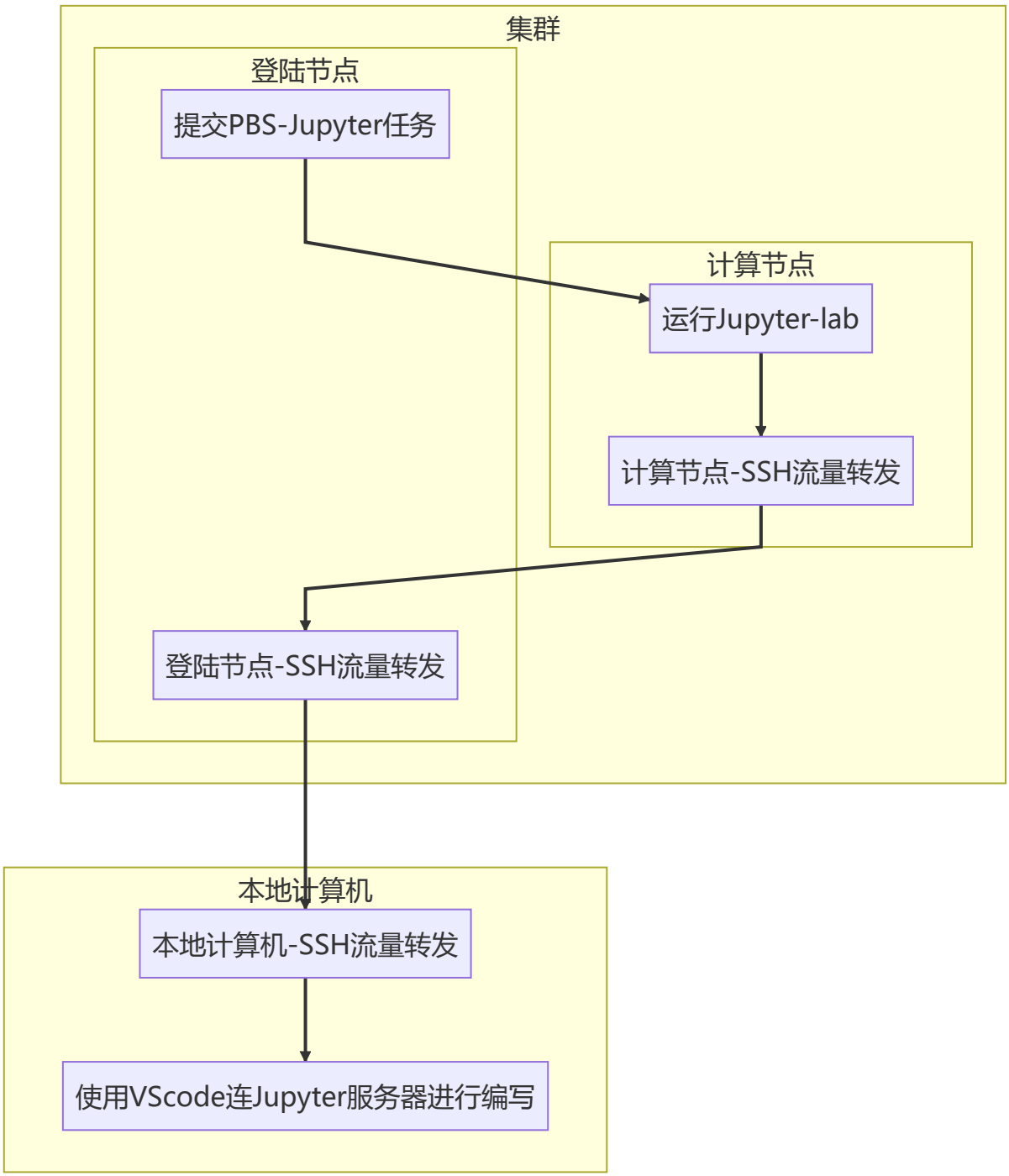
PBS脚本组成实例



计算实例——Han常用的Jupyter 云计算

完整全流程，适用于长时间计算

1. 思路：



拆解工作流可得到三个任务：

- 使用 PBS脚本提交Jupyter任务
- 多层ssh流量转发

- 本地vscode连接

现一一说明

2. 使用 PBS脚本提交Jupyter任务 `jupyter.pbs`

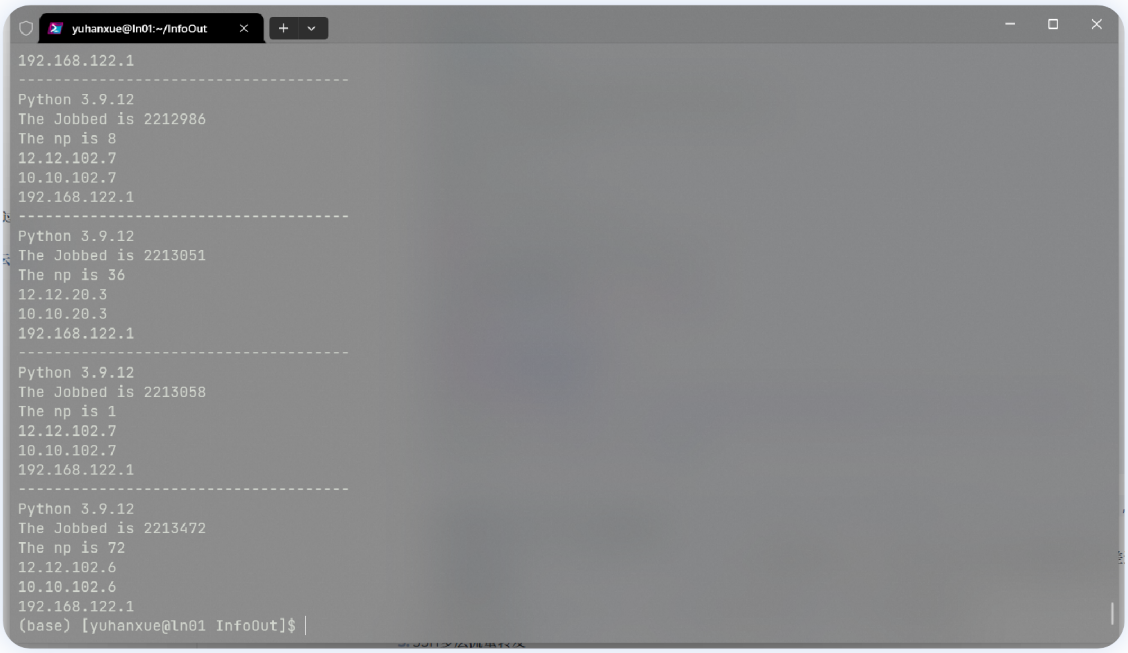
```
#pbs.sh
#!/bin/bash
#PBS -N jupyter
#PBS -o /lustre/home/yuhanxue/InfoOut/jupyte.out
#PBS -e /lustre/home/yuhanxue/InfoOut/jupyter.out
#PBS -l nodes=1:ppn=8
#PBS -q fat
#PBS -V
#PBS -S /bin/bash
#PBS -l walltime=1000:00:00
echo "-------------------------------------"
cd /lustre/home/yuhanxue/code/jupyter
conda activate pynb
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
python -V
echo "The Jobbed is $id"
NP=`cat $PBS_NODEFILE|wc -l`
echo "The np is $NP"
ip addr | awk '/^[0-9]+: / {}; /inet.*global/ {print gensub(/(.*)\/(.*)/,
"\\1", "g", $2)}'

jupyter-lab --no-browser --port=20260
```

- `ip addr | awk '/^[0-9]+: / {}; /inet.*global/ {print gensub(/(.*)\/(.*)/, "\\1", "g", $2)}'` 是为了输出网络信息，便于之后ssh流量转发
- `jupyter-lab --no-browser --port=20260` 无浏览器启动jupyter lab并运行在20260端口(你最好换个差远点的的端口，以免和我撞车)

  运行命令 `qsub jupyter.pbs`，在 `qstat` 查询，查询 `queue` 一栏为 `R` 即运行成功

3. SSH多层流量转发

- 首先先查询输出文件找到计算节点IP地址（以10.10开头）



- 在这里就可以看到最后一次日志的计算IP是10.10.102.6
- 在本地计算机的termianl上（Powershell）使用

```
ssh -t -t $USER@10.130.2.222 -L $PORT:localhost:$PORT ssh $IP -L
$PORT:localhost:$PORT
```

- `$USER` 集群用户账户名
- `$IP` 上一步计算节点的IP地址
- `$PORT` 自定的Jupyter运行端口

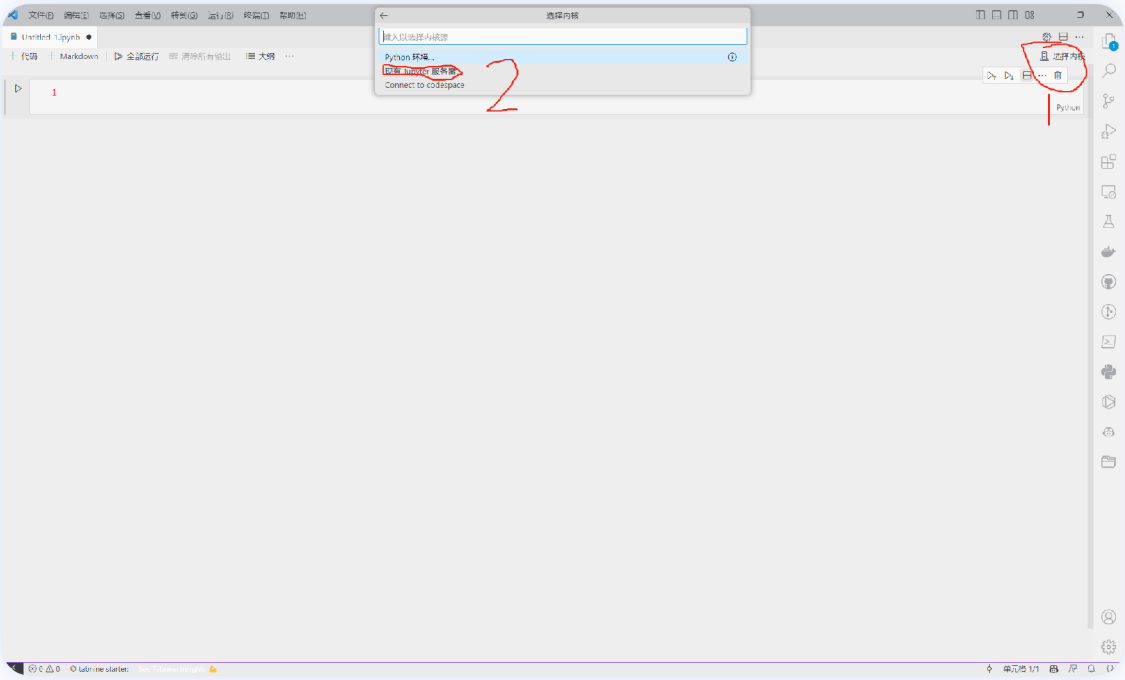
在我们这种情况即为

```
ssh -t -t yuhanxue@10.130.2.222 -L 20260:localhost:20260 ssh 10.10.102.6 -
L 20260:localhost:20260
```

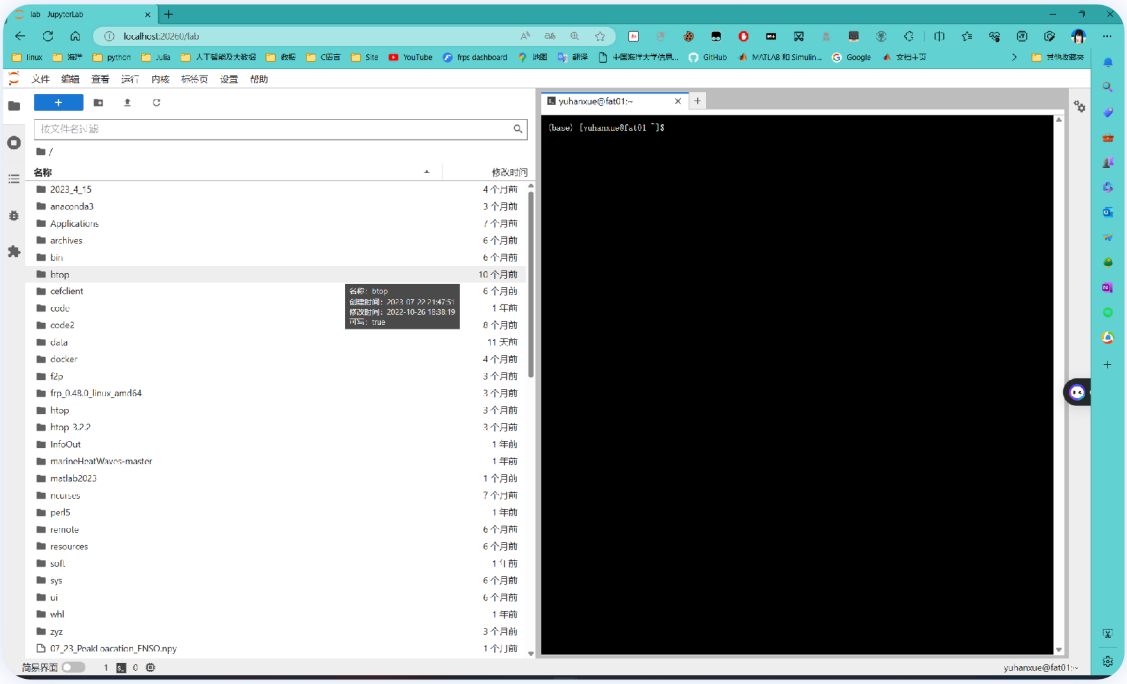
运行会让你输密码，出现shell即为成功，浏览器打开 `http://localhost:20260/` 试试能否打开

4. VScode连接

- VS code 笔记本支持远程Jupyter，用VS code打开你需要用HPC运行的笔记本



- 选输入URL，输入 `http://localhost:20260/`，稍等之后再选择Jupyter内核即可



5. 关闭任务（🌟）一定要记得，不然就花钱如流水了（悲
    1. qstat 查找任务ID
    2. qdel 任务ID
    3. qstat 确定任务关闭（不显示或 `queue` 为 `C` ）即可
    4. 在登录节点运行 `pkill -u $USER` 其中 `$USER` 换为自己的用户名，这个命令用来kill自己在登录节点的所有进程

便捷快速连接Jupyter（利用集群调试节点），适用于短时间调试和计算

集群提供免申请的调试节点，可以直接远程连接这个节点（fat01，10.10.102.1）进行计算

1. 在本地终端执行（SSH流量转发）

```
ssh -t -t $USER@10.130.2.222 -L 20260:localhost:20260 ssh 10.10.102.1 -L
20260:localhost:20260
```

   - `$USER` 集群用户账户名

2. 进去之后执行（启动Jupyter）

```
conda activate $CONDA_PYENV;jupyter-lab --no-browser --port=20260
```

   - `$CONDA_PYENV` 为需要的conda环境名，账户 `yanghaocheng` 中我安装的主要计算环境名为 `han` ，即YHC想用应该使用：

```
conda activate han;jupyter-lab --no-browser --port=20260
```

3. 重复上面 `VScode连接` 一步，进行代码调试。
4. 如果需要暂停按下 `Ctrl+C` ，然后输入 `pkill -u $USER` 其中 `$USER` 换为自己的用户名